

Advanced Natural Language Processing

Discussion 4: Syntax Semantics and Statistics

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes – Course year 2026-2027

Discussion Lecture: Syntax Semantics

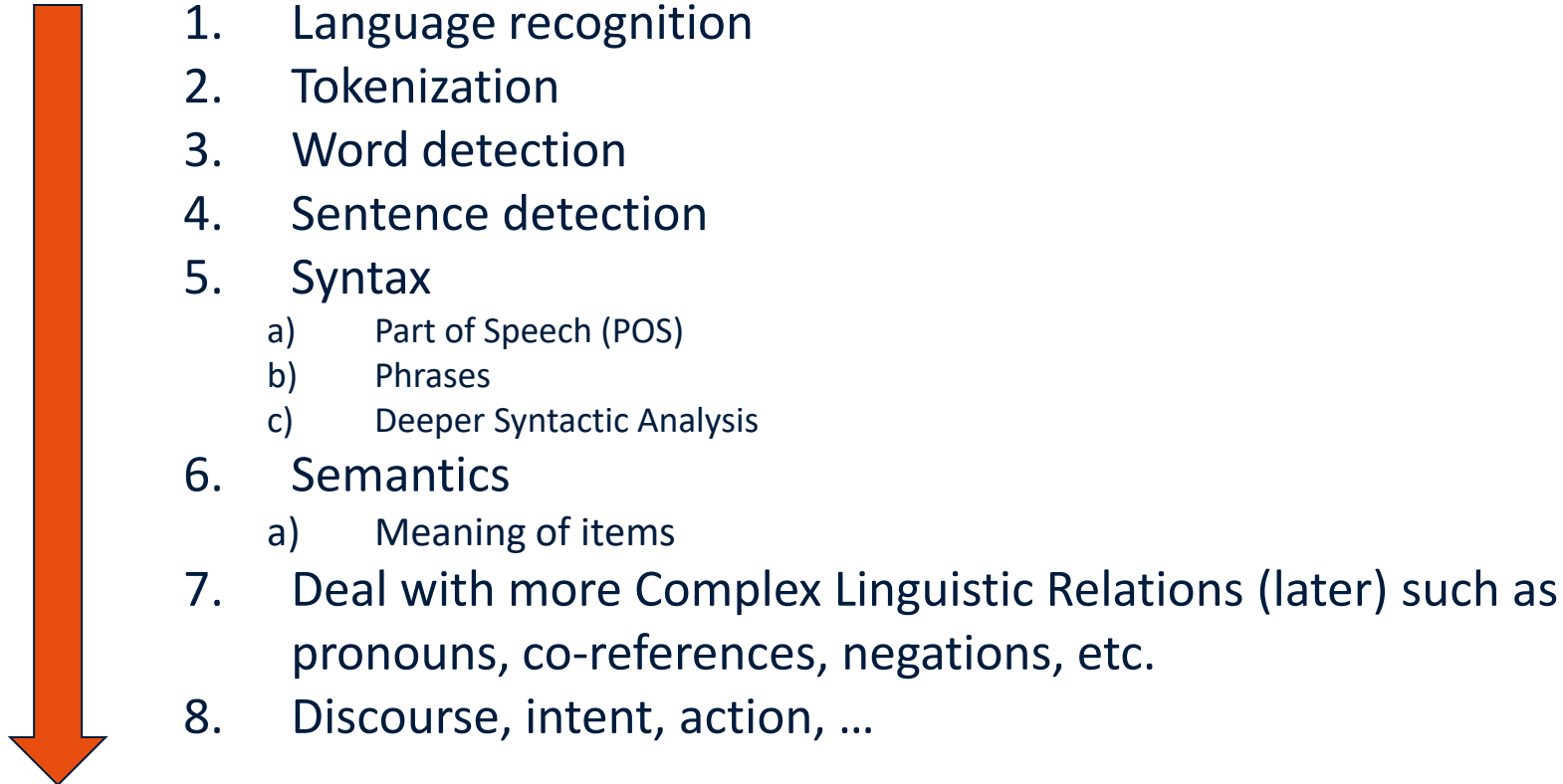


Maastricht University

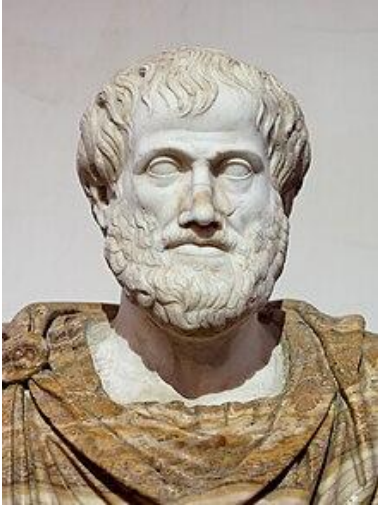
Learning Goals Today

- Understand Syntactic Structures
- Understand Part of Speech (POS) tagging
- Understand FSG, CFG and CSG
- Understand Chomsky's hierarchy
- Understand Probabilistic Grammars
- Understand Dependency Grammars
- Understand co-reference and pronoun resolution
- Understand negation handling

Traditional Linguistic Pipeline in more Detail



Linguistics and syntactic structures



Aristotle (384-322 BC)
Rhetoric, nominal and verbal words in Greek language

Dionysius Thrax (170-90 BC):
Techne Grammatica, “parts of speech”, focus on rhetoric, “good language”

Also early linguistics in India and China



Ferdinand de Saussure (1857 -1913)
Course in General Linguistics

Start of the science of linguistics
Phonology, phonetics, systems of *signs*

Descriptive linguistics

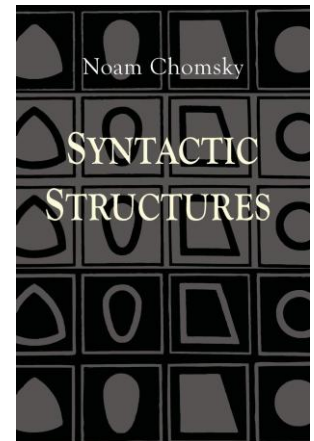


Noam Chomsky (1928-)
Syntactic Structures

Start of the modern linguistics, generative grammars, synthetic languages

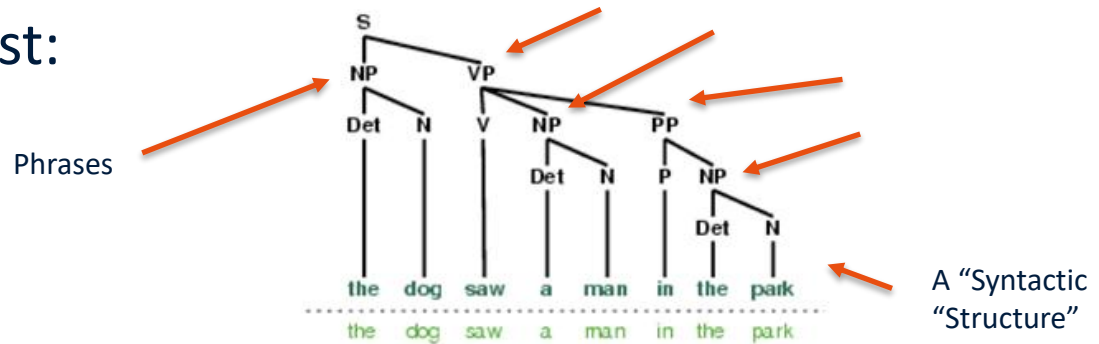
Universal Grammar; the idea of human innate grammatical knowledge

Formal language theory, theoretical computer science (compilers), artificial intelligence



Sentences and Phrases

- A sentence is made up from a string of words and punctuations.
- In a sentence, sub-sentences can be identified: aka phrases.
- Five types of **phrases** exist:
 - Noun
 - Verb
 - Adjective
 - Adverb
 - Prepositional



*She sings **very loudly**.*

- Head = **loudly** (an adverb)
- Whole phrase = **very loudly** → this is an **adverb phrase** (because it's headed by an adverb).

Object, Predicate (verb), Subject

<i>Sentence</i>	<i>Subject</i>	<i>Verb/Predicate</i>	<i>Object(s)</i>
The dog chased the ball .	The dog	chased	the ball (direct object)
She gave me a gift.	She	gave	me (indirect) + a gift (direct)
He is tired.	He	is tired (predicate; no object)	—

Subject = Noun that the sentence is about. The **Verb** expresses the action or state of being. The **Object** is what receives the action of the verb.

POS: Part of Speech

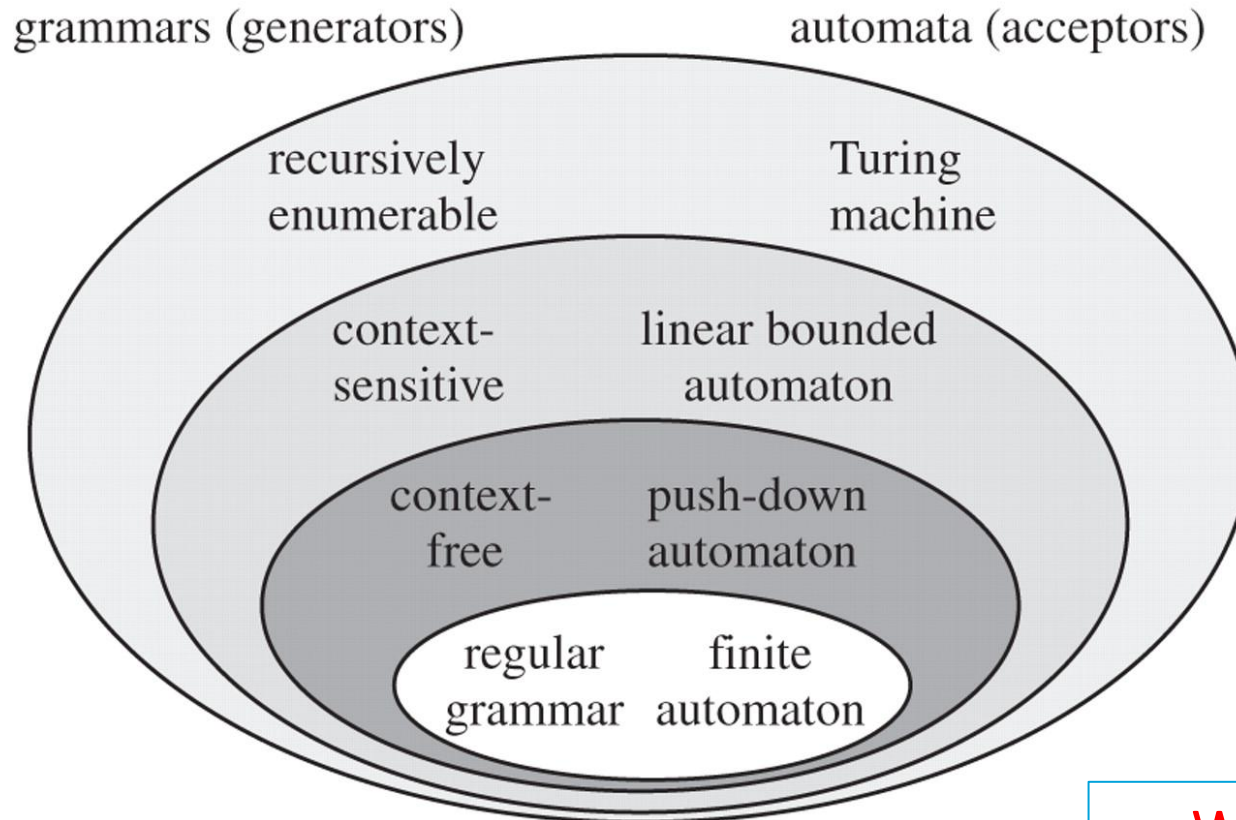
Noun Pronoun Verb Adjective Adverb Conjunction Preposition Article Interjection

Andrew and Maria thought their jobs were secure after the rancorous argument with the customer , but alas ! Bad news is fast approaching them , especially after they viciously insulted the customer on social media .

Part-of-Speech (POS) Tagging: In linguistics, part-of-speech tagging (POS tagging or POST), also called **grammatical tagging** or **word-category disambiguation**, is the process of marking up the words in a text (corpus) as corresponding to a particular part of speech, based on both its definition, as well as its context —ie. relationship with adjacent and related words in a phrase, sentence, or paragraph.

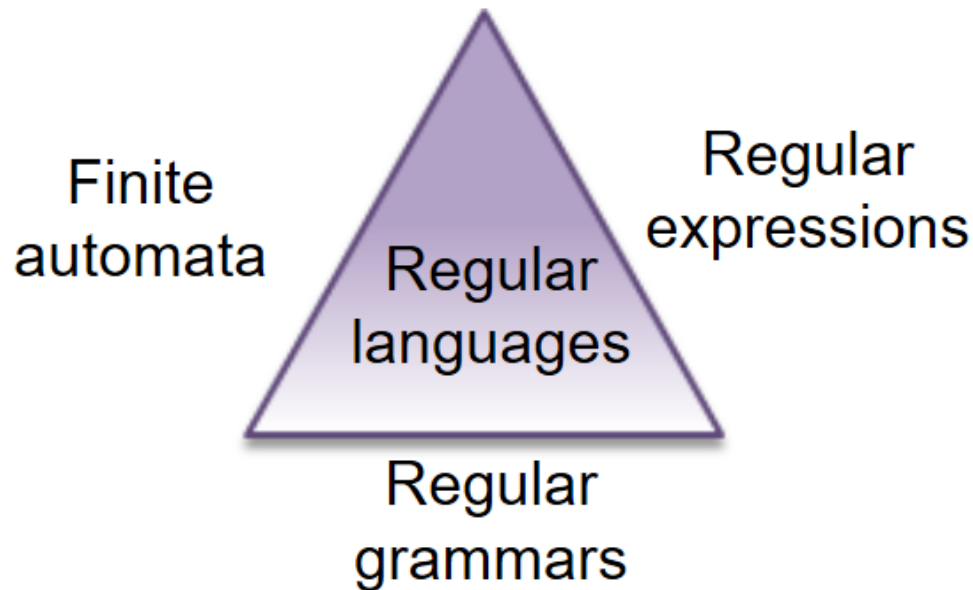
Schools commonly teach that there are 9 parts of speech in English: **noun, verb, article, adjective, preposition, pronoun, adverb, conjunction, and interjection**. However, there are clearly many more categories and sub-categories.

Chomsky's hierarchy



Where would
Natural language fit
in this hierarchy?

So, there are 3 ways to describe regular languages



Three equivalent ways of describing regular languages.

Natural Language is not a regular language

- Regular grammars cannot express languages that express **recursive center embedding**: a non-terminal is rewritten as itself surrounded by (non-empty) strings: $A \Rightarrow \alpha A \beta$.
- As a result, a language can be generated by a FSA as long as it does not have **center-embedded** recursions.
- Intuitively this is because grammar rules in which the non-terminal symbols are always on the right or on the left edge of a rule can be processed iteratively and not recursively.
- If one models center-embedded recursion with a FSA then it needs to be modeled as an iterative process. This will always result in either a very complex deep process or into limited recursion.

Context Free Grammars (CFG)

- CFG are one of the most common use mathematical system for modeling constituent structures English and in other languages.
- **CFG allow recursion.**
- CFG are also known as Phrase-Structure Grammars. Formalist is Backus-Naur form (BNF).
- Ideas of CFG goes back to Wilhelm Wundt (1900), but was not formalized until the work done independently by Chomsky (1959) and Backus (1959).
- CFG = rules (or productions) that express how symbols of a language can be grouped or ordered PLUS a lexicon of words and symbols.

Some example CFG for a Noun Phrase

Nouns can stack (e.g.,
cargo flight schedule
change)

Definitions:

NP (Noun Phrase): A phrase that behaves like a noun.

Det (Determiner): Words like a, the, some, this.

Noun: A lexical noun (e.g., flight, doctor, book).

ProperNoun: A proper name (e.g., Smith, Paris).

Nominal: A noun plus optional additional nouns.

NP $\rightarrow$ Det Nominal

NP $\rightarrow$ ProperNoun

Nominal $\rightarrow$ Noun

Nominal $\rightarrow$ Nominal Noun

Det $\rightarrow$ a

Det $\rightarrow$ the

Noun $\rightarrow$ flight

Noun $\rightarrow$ doctor

ProperNoun $\rightarrow$ Smith

Where is the recursion?

DP versus C-PS Parse

Dependency parse is like a family tree of word relationships. (“Who depends on whom?”)

Constituent parse is like a set of nested folders. (“Which words belong together as a phrase?”)

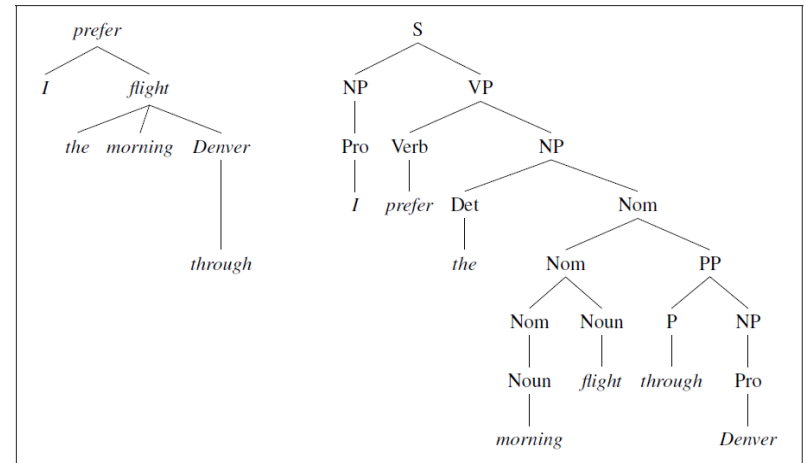


Figure 13.1 A dependency-style parse alongside the corresponding constituent-based analysis for *I prefer the morning flight through Denver*.

Feature

Focus

Units

Representation

Root of sentence

Used in

Dependency Parse

Word-to-word relationships

Words only

Flat, head-based

The main verb (e.g., *prefer*)

Modern NLP parsers (e.g., spaCy, UD)

Constituent (Phrase-Structure) Parse

How words group into phrases

Words + phrases (NP, VP, PP, etc.)

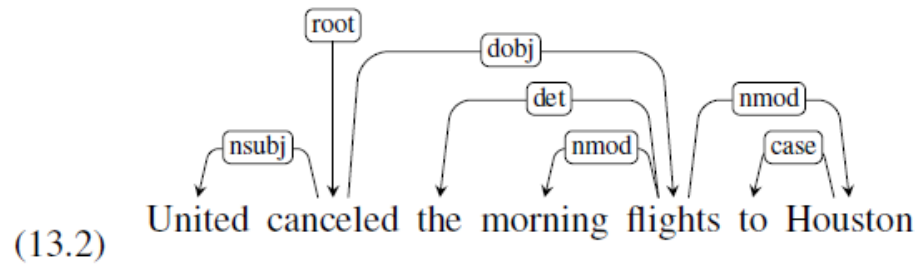
Hierarchical, phrase-based

The sentence node *S*

Formal syntax, grammars, CFGs

Example

Consider the following example sentence:

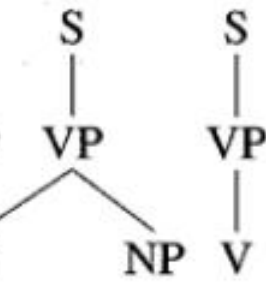
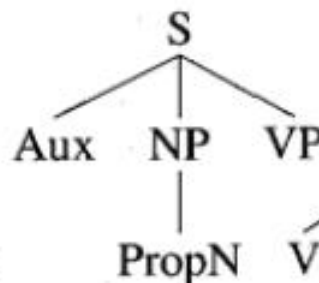
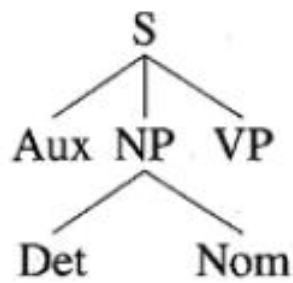
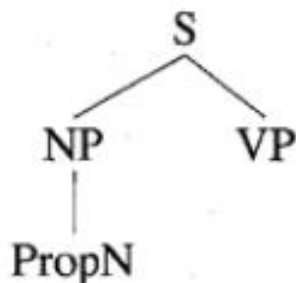
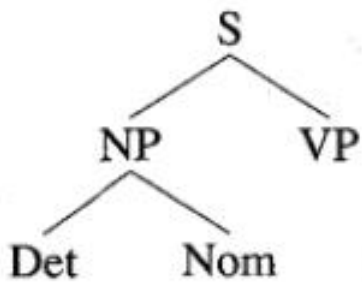
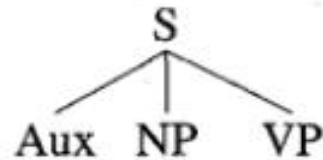
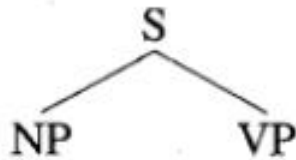


The clausal relations NSUBJ and DOBJ identify the subject and direct object of the predicate *cancel*, while the NMOD, DET, and CASE relations denote modifiers of the nouns *flights* and *Houston*.

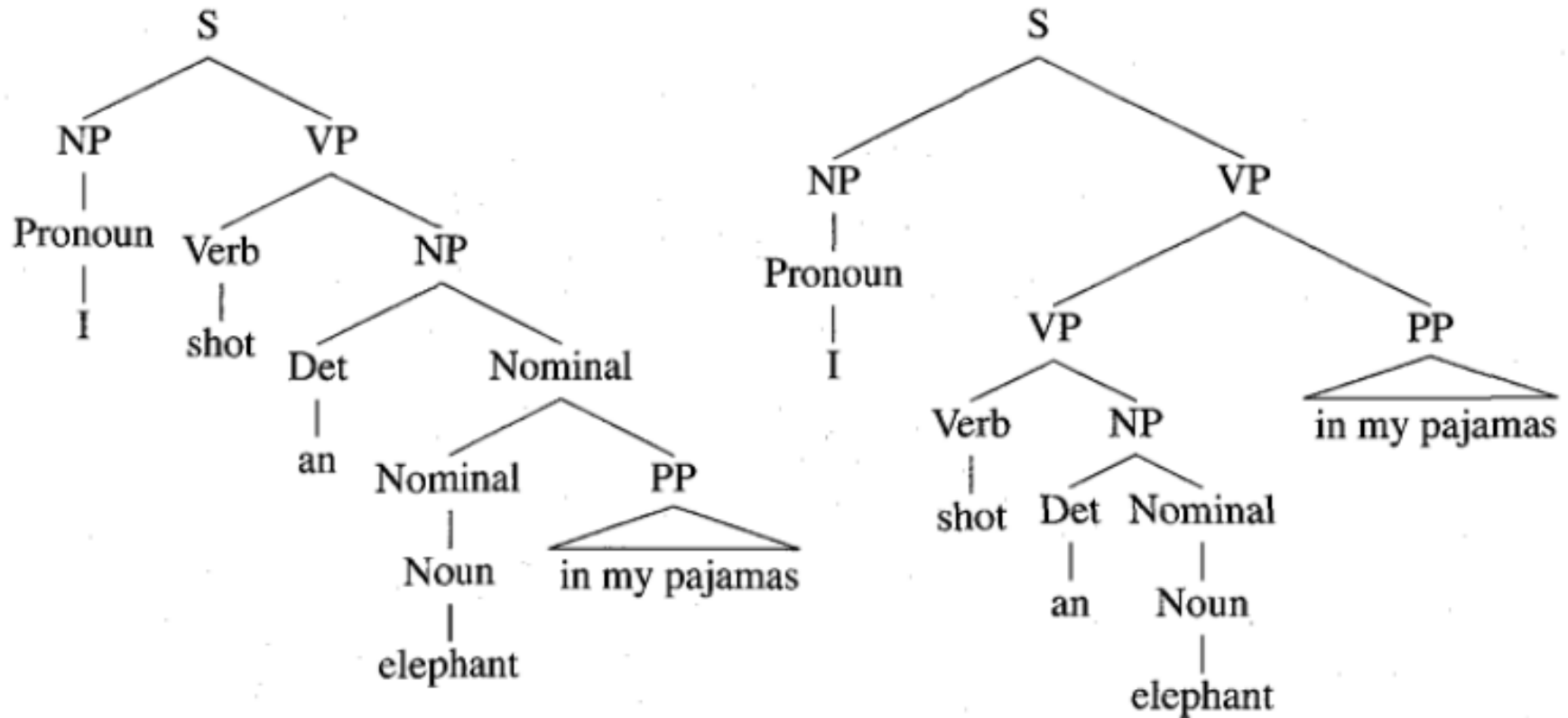
<i>Dependency label</i>	<i>Meaning</i>	<i>Example here</i>
root	Main verb of the sentence	canceled
nsubj	Subject performing the action	United
dobj	Direct object receiving the action	flights
det	Determiner modifying a noun	the
nmod	Nominal modifier (often part of a PP)	morning / Houston
case	The preposition linking a PP to a noun	to

An Expanding Top-Down Search Space

S



Two Parses for an Ambiguous Sentence



PCFG Definition

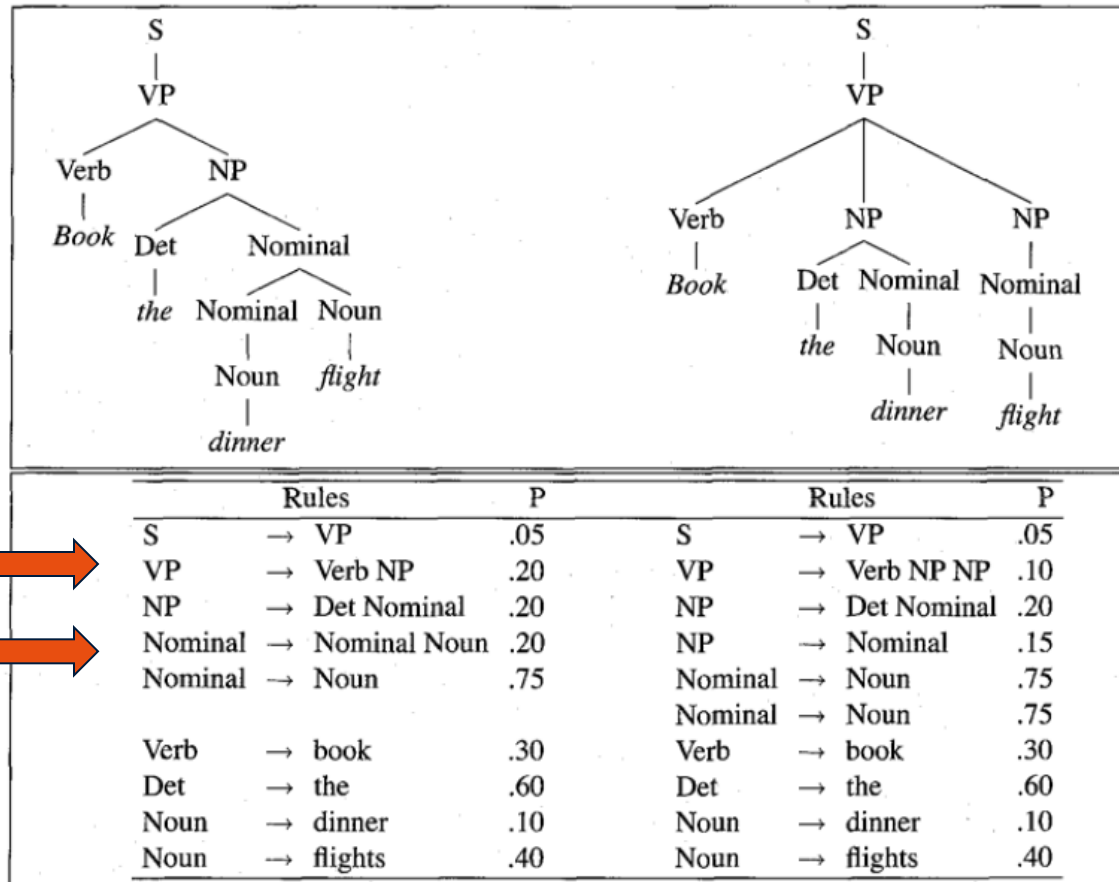
- N a set of **non-terminal symbols** (or **variables**)
- Σ a set of **terminal symbols** (disjoint from N)
- R a set of **rules** or productions, each of the form $A \rightarrow \beta [p]$,
where A is a non-terminal,
 β is a string of symbols from the infinite set of strings $(\Sigma \cup N)^*$,
and p is a number between 0 and 1 expressing $P(\beta|A)$
- S a designated **start symbol**

$$\sum_{\beta} P(A \rightarrow \beta) = 1$$

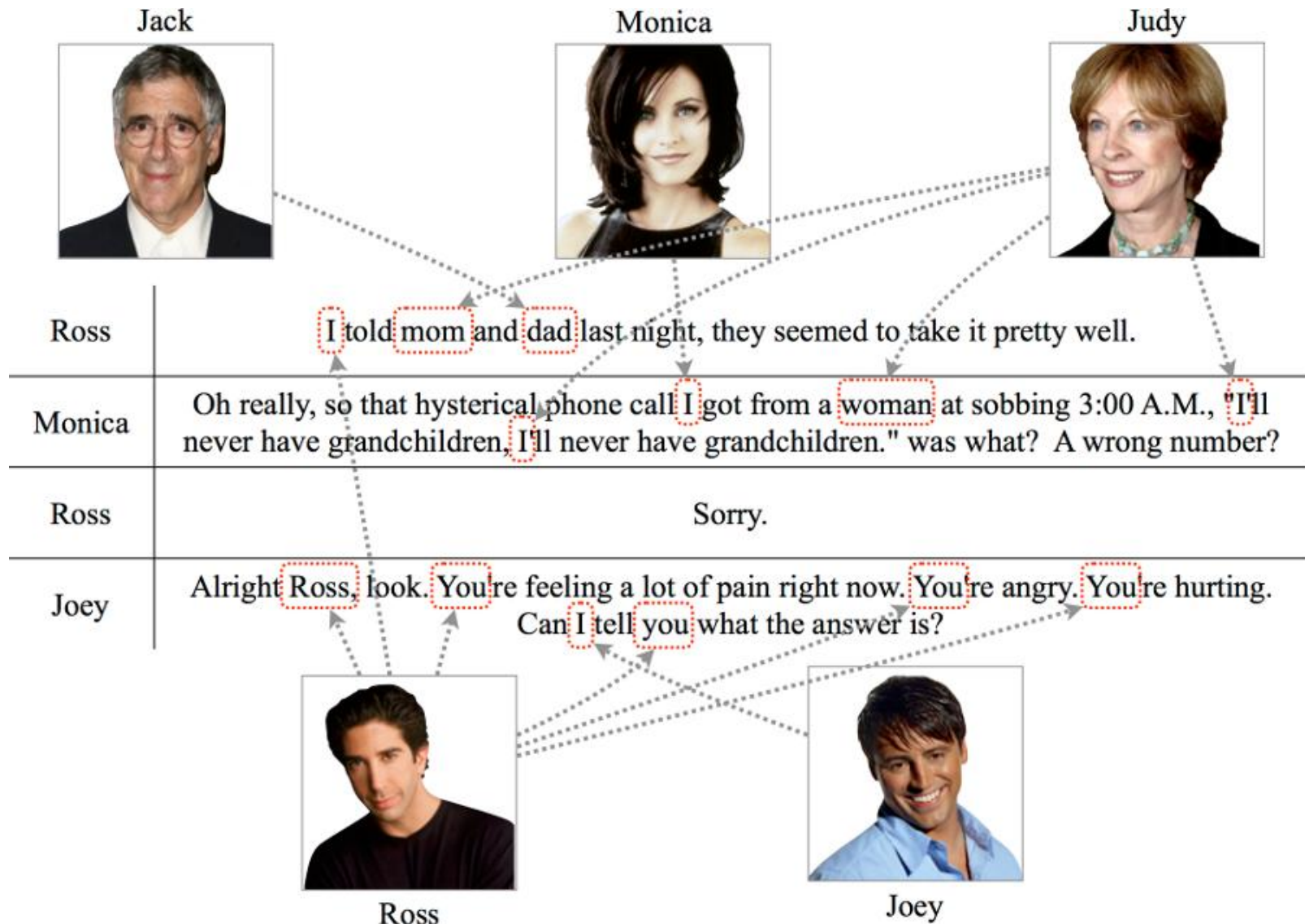
PCFG -2

$S \rightarrow NP VP$	[.80]	$Det \rightarrow that [.10] \mid a [.30] \mid the [.60]$
$S \rightarrow Aux NP VP$	[.15]	$Noun \rightarrow book [.10] \mid flight [.30]$
$S \rightarrow VP$	[.05]	$\mid meal [.15] \mid money [.05]$
$NP \rightarrow Pronoun$	[.35]	$\mid flights [.40] \mid dinner [.10]$
$NP \rightarrow Proper-Noun$	[.30]	$Verb \rightarrow book [.30] \mid include [.30]$
$NP \rightarrow Det Nominal$	[.20]	$\mid prefer; [.40]$
$NP \rightarrow Nominal$	[.15]	$Pronoun \rightarrow I [.40] \mid she [.05]$
$Nominal \rightarrow Noun$	[.75]	$\mid me [.15] \mid you [.40]$
$Nominal \rightarrow Nominal Noun$	[.20]	$Proper-Noun \rightarrow Houston [.60]$
$Nominal \rightarrow Nominal PP$	[.05]	$\mid TWA [.40]$
$VP \rightarrow Verb$	[.35]	$Aux \rightarrow does [.60] \mid can [40]$
$VP \rightarrow Verb NP$	[.20]	$Preposition \rightarrow from [.30] \mid to [.30]$
$VP \rightarrow Verb NP PP$	[.10]	$\mid on [.20] \mid near [.15]$
$VP \rightarrow Verb PP$	[.15]	$\mid through [.05]$
$VP \rightarrow Verb NP NP$	[.05]	
$VP \rightarrow VP PP$	[.15]	
$PP \rightarrow Preposition NP$	[1.0]	

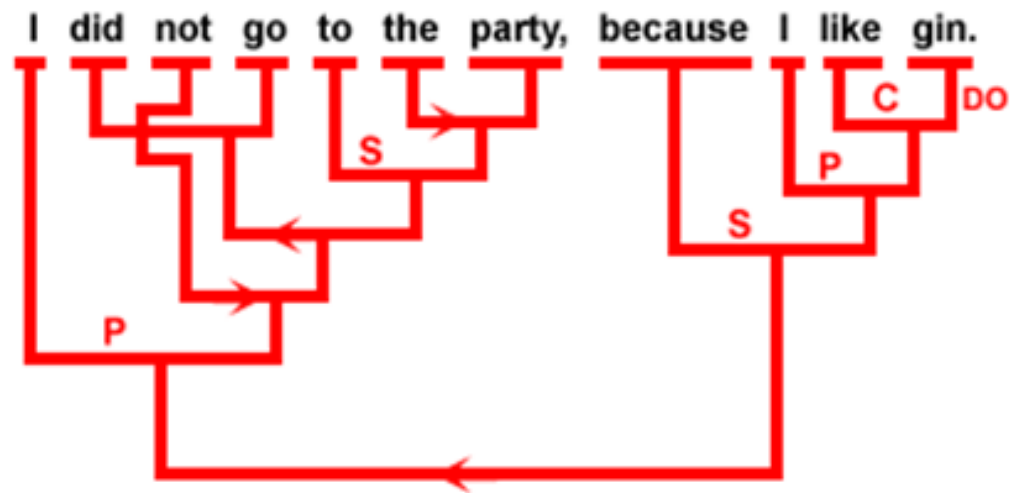
Disambiguating with statistics: “book a flight that serves dinner” versus “book a flight on behave of “the dinner””



Co-reference & Anaphora Resolution



impossible
impatient
unsafe
unknown
unemployed



Take Away

- The methods discussed in this course are traditional computational linguistics.
- Not really fault tolerant.
- Hard to create and maintain.
- Not very successful for real-world applications.

However, when we get to transformers, all the linguistic phenomena we discussed today will return and you will understand why transformers are as good as they are dealing with human language.

Questions on the lecture?

Now let's form groups of 4



Discuss Exercises Lecture Syntax and Semantics

Exercise 1

Slide 32 demonstrates recursive center-embedding in natural language: *"The luggage arrived"* → *"The luggage that the passengers checked arrived"* → *"The luggage that the passengers that the storm delayed checked arrived"*.

- Using formal language theory and the Chomsky hierarchy (Regular, Context-Free, Context-Sensitive, Recursively Enumerable), explain mathematically why a Finite State Automaton (FSA) or regular grammar cannot generate center-embedded structures of arbitrary depth.
- Why does recursive center-embedding require at least a Context-Free Grammar (CFG) / push-down automaton?
- Consider the simplified grammar L_0 from the lecture:
 - $S \rightarrow NP VP$
 - $NP \rightarrow Det Nominal \mid Pronoun$
 - $Nominal \rightarrow Noun \mid Nominal Noun$
 - $VP \rightarrow Verb NP \mid Verb$
 - Given the lexicon: $Det \rightarrow a$, $Noun \rightarrow flight \mid morning$, $Pronoun \rightarrow I$, $Verb \rightarrow want$.
 - Draw or write out the step-by-step derivation for the sentence: *"I want a morning flight"*.
Where does noun stacking occur, and how does the CFG rule accommodate it?

Exercise 2

Compare Phrase-Structure (Constituency) grammars and Dependency grammars in their treatment of sentence structure and structural ambiguity.

- Contrast the structural representation of a sentence in a Constituency parse versus a Dependency parse:
 - What serves as the root node in each framework?
 - How do the two paradigms differ regarding non-terminal phrasal nodes (e.g., *NP*, *VP*, *PP*)?
 - Why are dependency grammars more naturally suited for free-word-order languages (such as Czech or Dutch) than strict phrase-structure grammars?
- Consider the structurally ambiguous sentence: *"I saw the man on the hill with a telescope"*. Explain the difference between **attachment ambiguity** and **coordination ambiguity**.
- In a Probabilistic Context-Free Grammar (PCFG), the probability of a parse tree is computed by multiplying the probabilities of all independent rule expansions used. State the primary independence assumption made by a PCFG rule, and explain why this independence assumption fails to capture lexical preferences between specific verbs and prepositions

Exercise 3

Complex Discourse Phenomena (Coreference, Heuristics, and Negation Scope). Beyond sentence-level syntax, systems must resolve non-local discourse relations, including pronominal anaphora and negation scope.

- In the Hobbs (1978) algorithm and classical knowledge-poor anaphora resolution systems, candidates are filtered and ranked using grammatical constraints and salience heuristics.
 - List three agreement constraints (e.g., gender, number, person) used to prune invalid candidates.
 - If two surviving candidate nouns both satisfy all agreement checks, what ordering and structural heuristics (e.g., grammatical role, recency) determine which antecedent is chosen?
- Distinguish between **morphological negation** (e.g., *impossible*, *unemployed*) and **syntactic negation** (e.g., *denials*, *rejections*, *imperatives*).
- In the sentence: "*The audio in this TV is not very good*":
 - Why do simple fixed sliding windows and n-grams fail to reliably determine the exact scope of the negation?
 - Explain how traversing a **dependency grammar parse tree** allows an NLP system to locate the precise subtree/dependent modified by the negation marker (neg).

Discussion Lecture: Syntax Semantics



Maastricht University

Teaching note: Use today's grammar formalisms (CFG, dependency grammar, negation scope) as concrete cases for the abstract questions ahead.

On Chomsky's Hierarchy: Does Language Have a Grammar, or Do We?

Formal language theory tells us that finite state automata cannot handle center-embedding, so we moved to context-free grammars. Then Dutch and Swiss German produced cross-serial dependencies that are not context-free either.

If natural language keeps escaping the formalism we choose for it, what is the formalism describing: the language, the limits of our tools, or the limits of the human sentences we are willing to call grammatical?

Is formal syntax a discovery about how language is built, or an engineering scaffold that language constantly outgrows? (Keep this question in mind for the Transformer lectures.)

On Constituency vs. Dependency: Is Structure Discovered or Imposed?

A constituency grammar and a dependency grammar assign two different structures to the same sentence. Both are consistent with every observation; both support parsers that work. They cannot both be “the” structure of the sentence.

Is syntactic structure something that exists in the sentence and is discovered by the grammarian, or something the grammarian imposes because a purpose (translation, negation scope, free word order) makes one description more useful than another?

If two incompatible theories fit the data equally well, what would it take to decide between them, and does NLP need to decide at all?

On Negation: Is Logic Part of Language, or a Layer on Top of It?

A dependency parser gets negation right by rule and breaks on almost everything else. A large language model gets almost everything else right and still treats “A is not B” much like “A is B”.

Is negation a linguistic phenomenon or a logical one? If a model learns language purely from how it is used, should we expect it to learn logic at all, or is logic a different kind of thing that usage only hints at?

When we moved from symbolic trees to continuous vectors, we chose what to be good at and what to be bad at. Was that a trade-off anyone made consciously, and who is accountable for its consequences?

Tutorial 04: Syntax and Semantics



Maastricht University

Questions?



Maastricht University

Break



Discussion Lecture: Statistical NLP

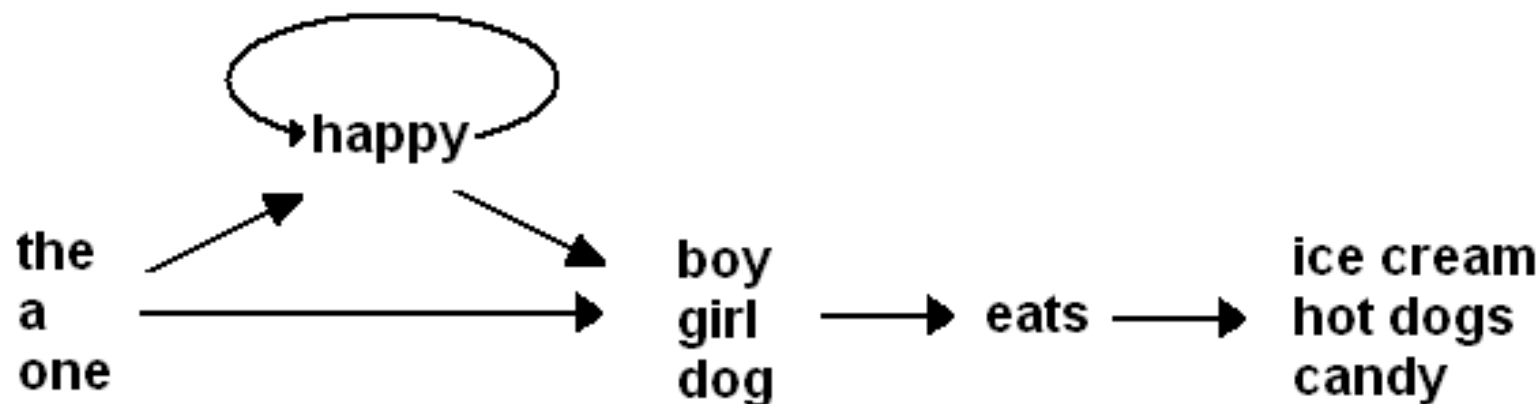


Maastricht University

Learning Goals

- Understand which, when and how statistical pre-processing methods can improve text-mining quality.
- Understand Naïve Bayes, Hidden Markov Models (HMM), Entropy Modeling and Conditional Random Fields (CRF) in more details so you can apply them successfully for NLP tasks.

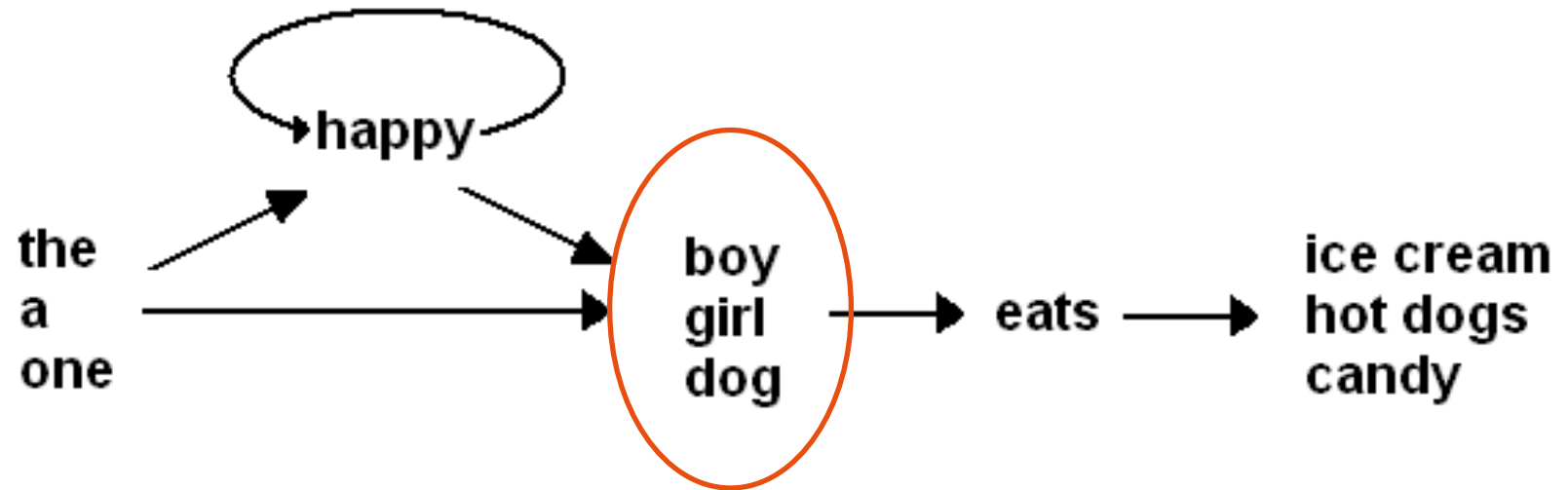
Last week: discrete Finite State Machines and Context-Free Grammars



Statistical parsing: transition probabilities

$$\sum_{\beta} P(A \rightarrow \beta) = 1$$

Language as a Sequence Model



Not only next-word prediction, but also POS-tagging, Named Entity Recognition (Semantic Role modeling) and other tagging c.q. classifications tasks in NLP, all the way to Abstracting, Machine Translation, Q&A and Chat Bots ...

Language as a Sequence Problem: would that work?

Writing English using randomly selected characters:

- Between A-Z:
`QWXVYBJSNKDPLTZURMOAEGCHIXFQWCBVYATRGDEJLHPZOXNMWK`
- With the **frequency of each letter** proportional to its commonness in the **English** language:
`EER ITHA NTHE RO AET S ETO NTE ORTA L R MATH ISE THA IOE NR`
- Where the frequency of each letter (and spaces) depends on its commonness in English and the likelihood of a **given letter following the one** that came before it (bi-grams):
`Thes e ar e wor ds t hat ma y sou nd qu ite ran dom but sti ll f orm a k
ind o f flo w.`
- With the conditional probability depending on **3 letters (trigrams or n-grams)** before the random new letter:
`The quiest of the compained and ther of the matation of you cande the
frospected the when.`

We can also do this with **randomly selected words** based on the **conditional probability of the previous words**:

- **1 word:** The man walked through the door and found a place to sit by the window.
- **2 words:** The man sat quietly by the window, looking out at the trees swaying in the breeze.
- **3 words:** The old man sat quietly by the window, watching the sun set behind the distant hills.

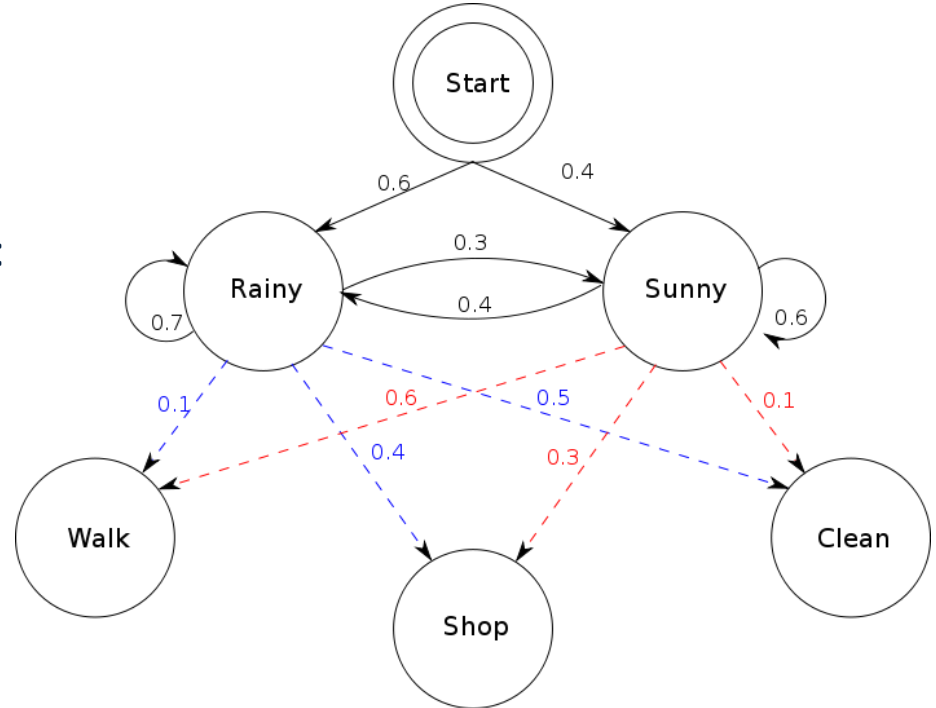


The Problem with n-gram (tokens) Naïve Bayes

- Still need to assume conditional independence of the labels
 - Overlapping features violate NB's independence assumption → probability estimates become distorted.
- New complications that are introduced:
 - Data sparsity:
 - number n-grams = $|V|^n$, with V Vocabulary size and n from n -gram.
 - There is never enough training data to cover all possibilities.
 - Not sequence-aware in the right way
 - Only backwards and limited context. Language also depends on what will come later.
 - Memory & computation cost: number of $P(n\text{-gram} | \text{class})$ is huge.
 - N-grams allow for correlation between word, but not between labels:
 - POS example: "Time flies like an arrow".
 - NB with trigrams can look at surrounding words for **flies**, but cannot reason that **flies's** tag should agree with its role given.
 - Fragile generalization:
 - If a new sentence contains an unseen trigram, NB falls back to smoothed prior, often too flat.

Example Revisited

- States = ('Rainy', 'Sunny') = Y_1, Y_2
- Observations = ('Walk', 'Shop', 'Clean') = O_1, O_2, O_3
- Start-Probabilities = {'Rainy': 0.6, 'Sunny': 0.4} = Π
- Transition Probabilities = {
 - 'Rain' : {'Rainy': 0.7, 'Sunny': 0.3},
 - 'Sun' : {'Rainy': 0.4, 'Sunny': 0.6},} = A
- Emission Probabilities = {
 - 'Rainy' : {'Walk': 0.1, 'Shop': 0.4, 'Clean': 0.5},
 - 'Sunny' : {'Walk': 0.6, 'Shop': 0.3, 'Clean': 0.1},} = B



How can we use this in NLP and Text Mining?

Observations = **Words** in Natural Language Sentences

States = **POS** type or **entity type** or **semantic meaning**

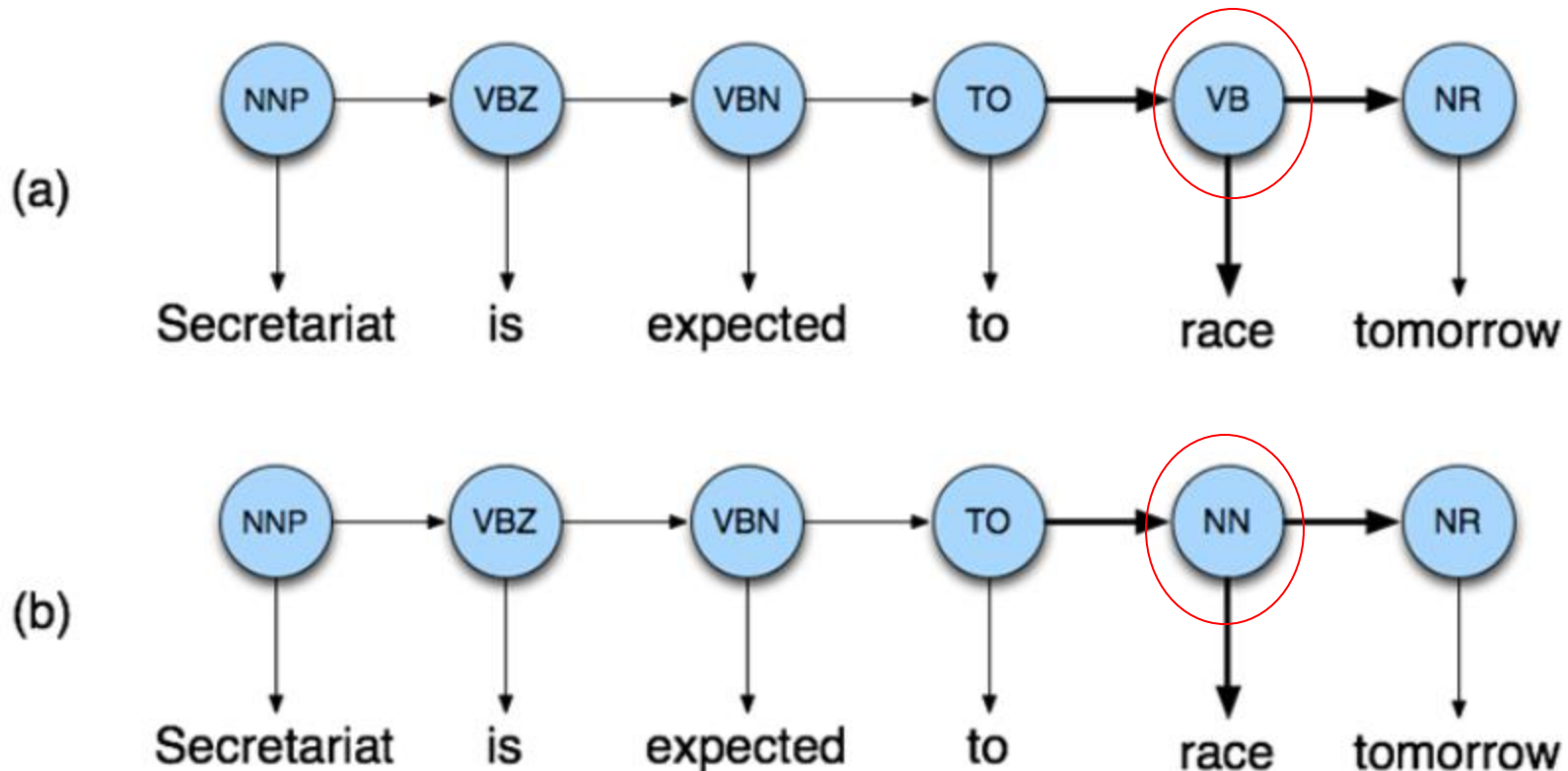
Use probabilities from HMM for linguistic disambiguation

HMM POS Taggers

The three (3) problems for an HMM translated to POS tagging:

1. Find the **probability** of an **observation sequence** = How likely is a test sequence generated by a particular HMM in POS.
2. Find the most likely **state** sequence that we can associate with a particular **observation** sequence = Find the most likely **POS tag** sequence that we can associate with a particular **word** sequence
3. Construct the HMM. **Estimate the parameters of the HMM** (state transition probability, observation matrix probability, initial state probabilities) were estimated from training data.

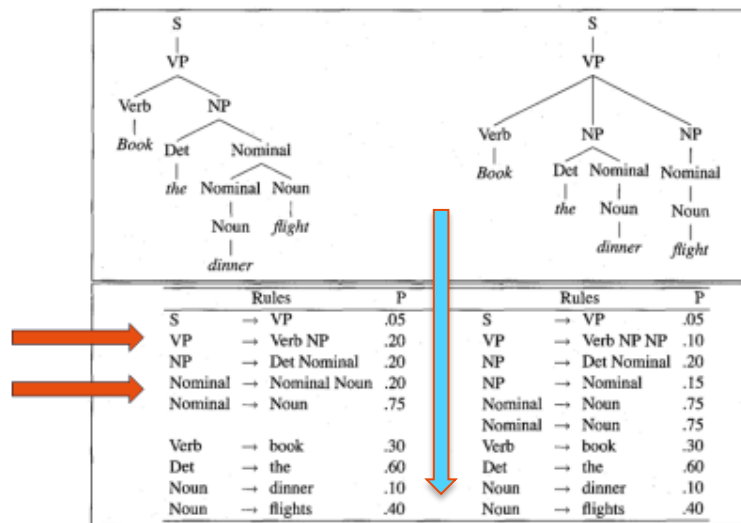
Disambiguating "race"



In this context, is RACE a Verb (VB) or a Nominal Noun (NN)?

Remember the PCFG approach ... this works much better!

Disambiguating with statistics: "book a flight that serves dinner" versus "book a flight on behave of "the dinner""



In PCFG the differences between the probabilities for different meaning are small. In the HMM approach, the differences are much larger, so the model is more certain.

Token	First Tag	Second Tag	Third Tag
Plastic	0.914 : nn	0.086 : jj	0.000 : np
signs	0.945 : nns	0.055 : vbz	0.000 : nn\$
work	0.632 : vb	0.367 : nn	0.001 : jj
around	0.929 : in	0.071 : rb	0.000 : vb
the	1.000 : at	0.000 : jj	0.000 : vb
clock	0.996 : nn	0.004 : rb	0.000 : jj
.	1.000 : .	0.000 : np	0.000 : nn

Viterbi with Log Probability

- Since the raw probability scores are usually very small in real examples (with many states).
- Therefore, we introduce a new concept: the joint log probability. The joint probability of the sequence of words found in the sentence will rank the probability the set of tags for a sentence.
- By taking the log of the summations instead of the raw multiplications, we avoid the probabilities to become smaller than what floating points in a computer can handle (floating point underflow). Additions are also computationally more efficient than multiplications of floating points.

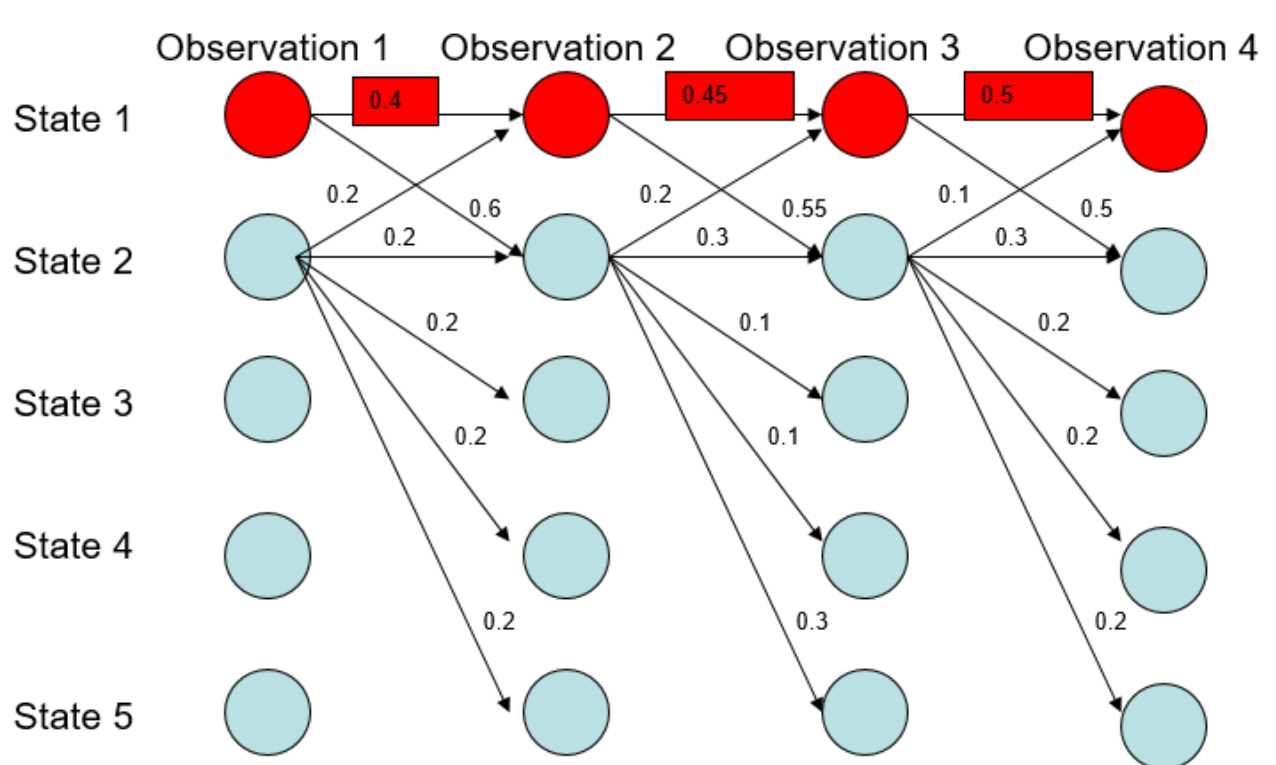
Log Probability	Tag Sequence
-80.460	Plastic_nn signs_nns work_vb around_in the_at clock_nn ._
-81.346	Plastic_nn signs_nns work_nn around_in the_at clock_nn ._
-83.624	Plastic_nn signs_nns work_vb around_rb the_at clock_nn ._

Limitations of HMM POS Taggers

The use of HMM to model natural language has of course its limitations:

- HMM's do not account for relationships with words that are not immediate neighbors.
 - **Long distance relationships** (which is one word in a first order HMM) do occur in natural language and cannot be modeled with a first-order HMM.
 - The probability of the tag of a word x is based on the **previous words'** tag as well as the tags of words from a text window centered on x (*FSG equivalent*).
- **HMM are still generative models**, so you need (plenty) data for every possible occurrence of combinations of words and POS tags, otherwise, it does not store it in the model (probability = 0) or if it occurs only once: bias ($p = 1$ for all such cases).

The Label Bias Problem with MEMM

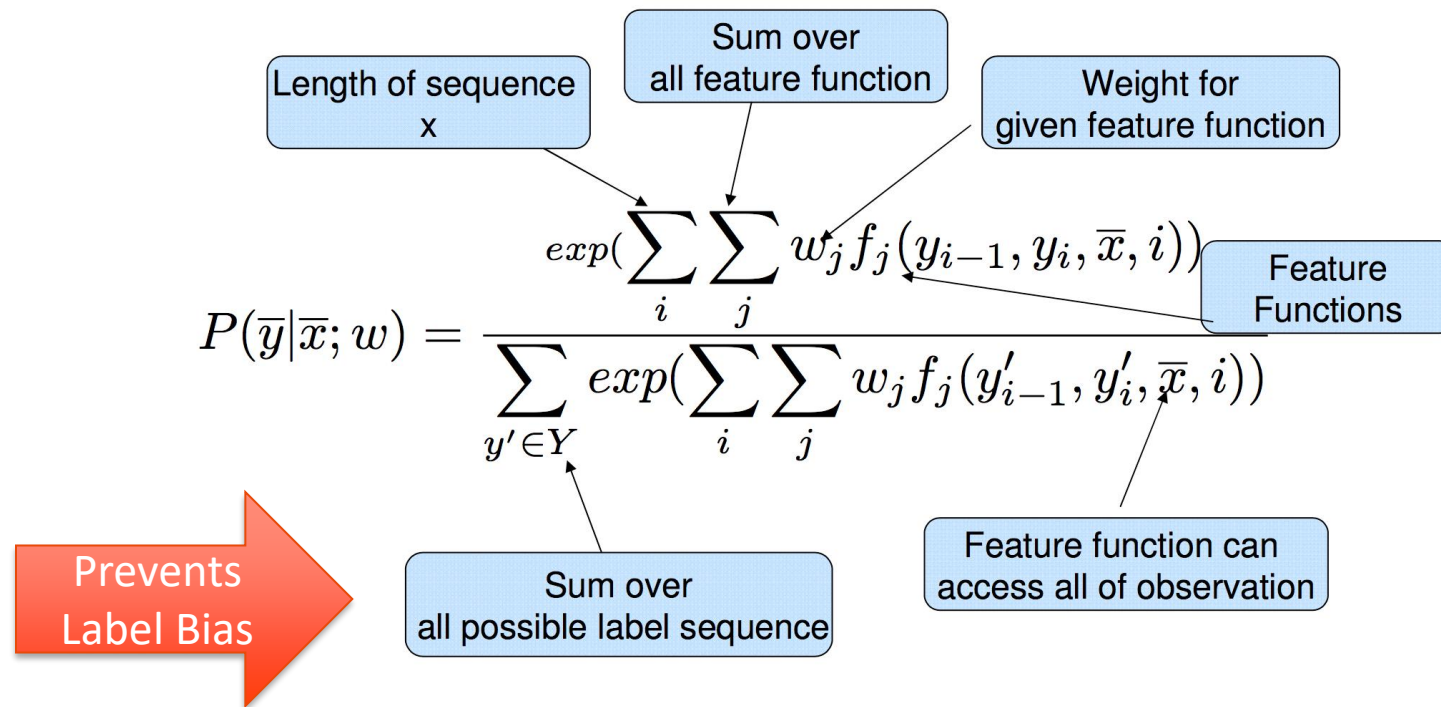


Most Likely Path: 1-> 1-> 1-> 1

- Although locally it seems state 1 wants to go to state 2 and state 2 wants to remain in state 2.

State 1 has only two transitions but state 2 has 5 = label bias problem. Go from local normalization to global.

CRF Formal Definition



Source: Wikipedia. See also Feldman pp. 142 for more explanations.

But CRFs Still have Limitations

CRFs are still a powerful structured prediction tool, but compared to modern deep-learning–based models, they have clear limitations:

- **Manual Feature Engineering:** CRFs rely on explicit features $f(y_{t-1}, y_t, x, t)$. You must design and extract these by hand: prefixes, suffixes, word shapes, gazetteers, etc. Deep models (RNNs, CNNs, Transformers) learn features automatically from raw text or embeddings.
- **Limited Representation Power:** CRFs are linear in the features:
 - $\text{Score}(x, y) = \theta \cdot F(x, y)$
 - They cannot easily capture:
 - Complex nonlinear interactions between context and labels,
 - Hierarchical or latent structure,
 - Long-distance dependencies (beyond what features encode).
 - Deep models learn rich, hierarchical representations (e.g., via self-attention).

More CRF Limitations

- **Context Window is Fixed:** Standard CRFs typically look at a small local window of words around position t . To incorporate long-distance context, you need explicit “skip” or “history” features, which quickly explode in number. Transformers capture context across the entire sentence (or even document) automatically.
- **Training & Scalability:** CRF training is $O(n|Y|^2)$ per gradient step (with forward–backward). With large tag sets or very long sequences, this is slow. Deep models may also be heavy, but they benefit from GPU acceleration and scale better with data size.
- **Handling Raw or Continuous Input:** CRFs assume discrete, sparse features. Incorporating embeddings or dense vectors directly is awkward (you must treat them as fixed inputs or pretrain them separately). Neural models can directly ingest embeddings and fine-tune them jointly.
- **Generative Capacity:** CRFs are discriminative: $P(y|x)$. They don’t model $P(x)$ or generate text; they only label given sequences. Deep models can be used as generative (e.g., seq2seq, language models) or discriminative.
- **Data Efficiency versus Representation:** with a small, well-featured dataset, CRFs can shine (fast, convex training). With massive data, deep models usually outperform (as we will see).

Overview of Natural Language Processing methods

Linguistics

Morphology:

- Lexicon's
- Inflection & stemming rules + exceptions.

Syntax:

- Finite State Grammar (FSG). Only look past words.
- Context Free Grammar (CFG). Look at past and future words.
- *Add probability* with a PCFG by estimating probabilities from large corpora of natural language.

Semantics:

- Dependency Grammar: captures forwards and backwards semantic relations.

Statistics

Syntax or semantics:

- HMM.
- MEMM – *add linguistic information* with feature functions. Label bias problem.
- CRF – *add linguistic information* with feature functions.

Deep Learning

Syntax or semantics:

- LSTM (looks only at past words – Word2Vec & GloVe)
- Bi-Directions LSTM (looks at past and future words – Elmo)
- Transformers & BERT: multi-headed attention

Where can they be used in NLP

Use a CRF whenever you want to assign a label to each token, and the correctness of one label depends on its neighbors. Especially for:

- Named entity recognition
- POS tagging
- Chunking
- Slot filling
- Sequence labeling tasks using BIO/BIOES schemes

Why still relevant

CRFs remain useful because they are data-efficient, computationally efficient, and structurally constrained sequence labeling models.

Unlike LLMs, CRFs do not require large annotated datasets or GPUs and can run on low-power devices with very low latency.

They allow explicit incorporation of domain knowledge and enforce valid label transitions, producing deterministic and interpretable output.

Although LLMs are superior for complex contextual reasoning, CRFs are still the preferred choice in low-resource, low-power, or high-regulation environments.

Questions on the lecture?

Discuss Exercises Lecture Syntax and Semantics

Exercise 1: HMM Parameter Estimation & Viterbi POS Disambiguation

Consider an HMM POS tagger evaluating the ambiguous word "*race*" in the sentence fragment: "Secretariat is expected to race tomorrow". The two candidate tags for "*race*" at position $t = 5$ are Verb (VB) and Noun (NN). The preceding tag at $t = 4$ for the word "*to*" was TO with a path probability of $\delta_4(\text{TO}) = 0.42$. You are given the following simplified model parameters:

- Transition probabilities: $a_{\text{TO},\text{VB}} = 0.80$, $a_{\text{TO},\text{NN}} = 0.05$
- Emission probabilities: $b_{\text{VB}}(\text{race}) = 0.05$, $b_{\text{NN}}(\text{race}) = 0.06$
- Using the Viterbi recurrence $\delta_t(j) = b_j(O_t) \cdot \max_i [\delta_{t-1}(i) \cdot a_{ij}]$, calculate the path probabilities $\delta_5(\text{VB})$ and $\delta_5(\text{NN})$.
- Notice that the emission probability of "*race*" as a noun (0.06) is actually higher than as a verb (0.05). Explain how the transition probabilities and syntactic context allow the Viterbi algorithm to correctly disambiguate "*race*" as VB.
- When scaling the Viterbi algorithm to long sentences with dozens of tags, explain why practical implementations convert products of probabilities into sums of log-probabilities ($\ell\delta_t, \ell a_{ij}, \ell b_j$). What hardware/floating-point problem does this prevent?

Exercise 2: MEMMs and the Label Bias Problem

Maximum Entropy Markov Models (MEMMs) replace fixed transition and emission lookup tables with a discriminative log-linear model conditioned on rich input features:

$$P(y_t \mid y_{t-1}, x) = \frac{\exp(\sum_k \theta_k f_k(x, y_{t-1}, y_t))}{Z(x, y_{t-1})}$$

- In the slide example for tagging "*race*", suppose the active feature weights $\sum \theta_k f_k$ evaluate to:
 - For VB: weights $0.8 + 0.01 + 0.1 = 0.91$
 - For NN: weights $0.8 - 1.3 = -0.5$
 - Using the softmax equation above, calculate the normalized local probabilities $P(\text{VB} \mid x)$ and $P(\text{NN} \mid x)$.
- Explain the **Label Bias Problem** inherent in MEMMs. Why does local normalization at each time step cause states with few outgoing transitions to dominate over states with many outgoing transitions, regardless of what the observation evidence supports?
- Provide an intuitive real-world analogy (e.g., navigating street corners or road networks) explaining how local step-by-step normalization differs from global sequence optimization.

Exercise 3: Generative vs. Discriminative Sequence Modeling (HMM → MEMM → CRF)

Review the theoretical evolution from generative HMMs to discriminative CRFs.

- Contrast HMMs and CRFs across the following architectural dimensions:
 - Model type: Which models the joint distribution $P(X, Y)$ and which models the conditional distribution $P(Y | X)$?
 - Input features: Why are HMMs strictly limited to discrete word tokens, whereas CRFs can incorporate arbitrary overlapping feature functions (e.g., capitalization, suffixes like -ing, regex digit patterns)?
- How does a Linear-Chain CRF mathematically eliminate the label bias problem found in MEMMs? (Hint: Address where the partition function $Z(x)$ normalizes the scores.)
- Despite the dominance of modern deep transformers (such as BERT or LLMs), list two engineering or practical reasons why CRFs remain the preferred choice for sequence labeling tasks in low-power, low-resource, or highly regulated production environments.

Discussion Lecture: Statistical NLP



Maastricht University

On the Markov Assumption: Can a Wrong Model Be the Right Tool?

A first-order Markov model assumes that the next state depends only on the current one. Yet subject and verb agree across whole clauses (“The captain of the three stranded fishing boats is arriving”), and a question is formed by moving a word to the front because of something at the end.

Does language have a Markovian memory, or are we imposing a memoryless chain on a hierarchical process because the chain is what we can compute?

HMMs were known to be wrong about language and were still the industrial standard for two decades. If a model can be false and still the best available tool, what is the relation between the truth of a model and its usefulness?

On Generative vs. Discriminative: Is Explaining the Input a Form of Understanding?

A generative model (HMM) tells a story about how the words came to be: tags produce words. A discriminative model (CRF) tells no story; it is a decision rule from words to tags. For tagging, the decision rule won.

Then the most successful language models in history turned out to be generative again: they do nothing but predict the next word. What changed: the data, the compute, or our idea of what a model is for?

Does a model that can explain how its input arose know more about language than one that only classifies it? Or is “explaining the input” just a longer way of memorising it?



On Commitment: Should a Reader Decide Now, or Wait for the End?

The label bias problem is, underneath the mathematics, a question about commitment: a model that normalises locally has to decide at every step and can never take a decision back. A globally normalised model defers every decision until it has seen the whole sentence.

Humans read incrementally and still get garden-path sentences wrong (“The horse raced past the barn fell”), then repair. Is understanding a local process that occasionally backtracks, or a global one that we only experience as local?

Transformers look at the whole sentence at once. Did they solve the problem of premature commitment, or did they abolish the idea that reading has an order at all, and what is lost if it does not?

Tutorial 05: Statistical NLP



Maastricht University

Questions?



Maastricht University